

Modül 12

Pipes

Veri Dönüşümleri

Modül:	12 / 30
Ders Sayısı:	6 ders
Seviye:	Kolay-Orta
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 12'ye Hoş Geldin

Pipe'lar template içinde veriyi dönüştürmek için kullanılır. `{{ date | date }}` gibi. Bu modülde built-in pipe'ları ve custom pipe yazmayı öğreneceğiz.

Bu modülün sonunda yapabileceklerin:

- ✓ Built-in pipe'ları (date, currency, number, async) kullanmak
- ✓ Türkçe locale ile çalışmak
- ✓ Custom pipe yazmak (truncate, fileSize, timeAgo)
- ✓ Pure vs Impure pipe farkını anlamak
- ✓ Pipe'ları ne zaman kullanmamak gerektiğini bilmek

Pipe Nedir? + Built-in'ler

Template'te veriyi dönüştürmenin Angular yolu.

Temel Syntax

```
<!-- value | pipe -->
{{ today | date }}           <!-- Tarih formatla -->
{{ price | currency }}      <!-- Para formatla -->
{{ name | uppercase }}      <!-- BÜYÜK HARF -->
{{ items | json }}          <!-- JSON string -->

<!-- Argument ile -->
{{ today | date:'short' }}   <!-- "5/19/26, 11:35 AM" -->
{{ price | currency:'TRY' }} <!-- "₺99.99" -->

<!-- Chain -->
{{ name | lowercase | titlecase }}
```

Built-in Pipes

Pipe	Ne Yapar	Örnek
date	Tarih formatla	{{ d date:"long" }}
currency	Para birimi	{{ p currency:"TRY" }}
number	Sayı format	{{ n number:"1.2-2" }}
percent	Yüzde	{{ p percent }}
uppercase	BÜYÜK	{{ s uppercase }}
lowercase	küçük	{{ s lowercase }}
titlecase	Title Case	{{ s titlecase }}
json	JSON string	{{ obj json }}
slice	Substring	{{ s slice:0:10 }}
async	Observable/Promise	{{ obs\$ async }}
keyvalue	Object iter	@for of (obj keyvalue)
i18nPlural	Çoğul/tekil	{{ n i18nPlural:msgs }}
i18nSelect	Switch	{{ gender i18nSelect:msgs }}

Date Pipe + Türkçe Locale

Tarih formatlama ve Türkçe için locale setup.

□ Date Pipe Format'ları

```
{{ today | date }}           <!-- May 19, 2026 -->
{{ today | date:'short' }}   <!-- 5/19/26, 11:35 AM -->
{{ today | date:'medium' }}  <!-- May 19, 2026, 11:35:00 AM -->
{{ today | date:'long' }}   <!-- May 19, 2026 at 11:35:00 AM -->
{{ today | date:'full' }}   <!-- Tuesday, May 19, 2026 at... -->
{{ today | date:'shortDate' }} <!-- 5/19/26 -->
{{ today | date:'mediumDate' }} <!-- May 19, 2026 -->
{{ today | date:'longDate' }} <!-- May 19, 2026 -->
{{ today | date:'fullDate' }} <!-- Tuesday, May 19, 2026 -->
{{ today | date:'shortTime' }} <!-- 11:35 AM -->
{{ today | date:'mediumTime' }} <!-- 11:35:00 AM -->

<!-- Custom format -->
{{ today | date:'dd/MM/yyyy' }} <!-- 19/05/2026 -->
{{ today | date:'HH:mm' }} <!-- 11:35 -->
{{ today | date:'EEEE' }} <!-- Tuesday -->
```

□□ Türkçe Locale Setup

```
// app.config.ts
import { registerLocaleData } from '@angular/common';
import localeTr from '@angular/common/locales/tr';
import { LOCALE_ID } from '@angular/core';

registerLocaleData(localeTr, 'tr');

export const appConfig: ApplicationConfig = {
  providers: [
    { provide: LOCALE_ID, useValue: 'tr' },
  ]
};
```

□ Türkçe Sonuçlar

```
{{ today | date }}          <!-- 19 May 2026 -->
{{ today | date:'long' }}    <!-- 19 Mayıs 2026 15:35:00 GMT+3 -->
{{ today | date:'EEEE' }}    <!-- Salı -->
{{ today | date:'MMMM' }}    <!-- Mayıs -->
{{ price | currency:'TRY' }} <!-- ₺99,99 (virgül!) -->
{{ n | number }}             <!-- 1.234,56 (Türk formatı) -->
```

async Pipe vs Signal

Observable'lar için async pipe. Ama artık signal varken?

❑ async Pipe — Klasik

```
@Component({
  template: `
    <!-- Observable'ı subscribe + unsubscribe otomatik -->
    <p>{{ user$ | async }}</p>

    <!-- as ile değişken -->
    @if (user$ | async; as user) {
      <p>{{ user.name }}</p>
    }
  `,
})
export class UserComponent {
  user$: Observable<User> = this.http.get<User>('/api/me');
}
```

❑ Signal — Modern

```
@Component({
  template: `
    <p>{{ user()?.name }}</p>

    @if (user(); as u) {
      <p>{{ u.name }}</p>
    }
  `,
})
export class UserComponent {
  user = httpResource<User>('/api/me');

  // user.value() → User | undefined
}
```

⚖ Hangisini Kullan?

Senaryo	Tercih	Sebep
Yeni kod	Signal	Modern, daha temiz
Mevcut Observable	async pipe	Zaten var, dönüştürmeye gerek yok

Complex stream	async pipe	RxJS operatörlerini kullan
Simple data	Signal	Daha az boilerplate

❏ async Pipe Tuzakları

⚠ Multiple Subscriptions

async pipe'ı template'te 2 kez kullanırsan, 2 subscription olur. İkinci defa async pipe yerine "as" ile alias kullan.

```
<!-- ❏ İki subscription -->
<p>{{ user$ | async | json }}</p>
<p>{{ (user$ | async)?.name }}</p>

<!-- ✓ Tek subscription -->
@if (user$ | async; as user) {
  <p>{{ user | json }}</p>
  <p>{{ user.name }}</p>
}
```

Custom Pipe Yazma

Kendi pipe'ini yazmak. truncate, fileSize, timeAgo, highlight.

❑ Temel Custom Pipe

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'truncate',
  standalone: true,
})
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit: number = 50, ellipsis: string = '...'): string {
    if (!value) return '';
    if (value.length <= limit) return value;
    return value.substring(0, limit) + ellipsis;
  }
}

// Kullanım
<p>{{ longText | truncate:100 }}</p>
<p>{{ longText | truncate:50:'...' }}</p>
```

❑ fileSize Pipe

```
@Pipe({ name: 'fileSize', standalone: true })
export class FileSizePipe implements PipeTransform {
  transform(bytes: number, precision: number = 2): string {
    if (!+bytes) return '0 Bytes';
    const k = 1024;
    const sizes = ['Bytes', 'KB', 'MB', 'GB', 'TB'];
    const i = Math.floor(Math.log(bytes) / Math.log(k));
    return parseFloat((bytes / Math.pow(k, i)).toFixed(precision)) + ' ' + sizes[i];
  }
}

<!-- 1536000 → "1.46 MB" -->
{{ file.size | fileSize }}
```

❑ timeAgo Pipe


```

@Pipe({ name: 'timeAgo', standalone: true })
export class TimeAgoPipe implements PipeTransform {
  transform(date: Date | string | number): string {
    const d = new Date(date);
    const seconds = Math.floor((Date.now() - d.getTime()) / 1000);

    const intervals: [number, string][] = [
      [31536000, 'yıl'],
      [2592000, 'ay'],
      [86400, 'gün'],
      [3600, 'saat'],
      [60, 'dakika'],
    ];

    for (const [secs, label] of intervals) {
      const count = Math.floor(seconds / secs);
      if (count > 0) {
        return `${count} ${label} önce`;
      }
    }

    return 'az önce';
  }
}

<!-- "3 saat önce", "5 dakika önce" -->
{{ post.createdAt | timeAgo }}

```

□ highlight Pipe

```

import { DomSanitizer } from '@angular/platform-browser';

@Pipe({ name: 'highlight', standalone: true })
export class HighlightPipe implements PipeTransform {
  private sanitizer = inject(DomSanitizer);

  transform(text: string, search: string) {
    if (!search) return text;
    const regex = new RegExp(search, 'gi');
    const highlighted = text.replace(regex, m => `<mark>${m}</mark>`);
    return this.sanitizer.bypassSecurityTrustHtml(highlighted);
  }
}

<!-- [innerHTML] gerekli çünkü HTML inject ediyoruz -->
<p [innerHTML]="text | highlight:searchTerm()"></p>

```

Pure vs Impure Pipe

Performance kritik: ne zaman impure kullanmak güvenli.

❏ Pure Pipe (Default)

Pure pipe: Sadece input referansı değişince yeniden çalışır. Aynı input için cache'lenir.

```
@Pipe({ name: 'double', standalone: true }) // pure: true (default)
export class DoublePipe implements PipeTransform {
  transform(value: number): number {
    console.log('Computing...');
    return value * 2;
  }
}

// Template
{{ count() | double }}
// count = 5 → "Computing..." → 10
// count = 5 → (cache) → 10 (yeniden compute yok)
// count = 10 → "Computing..." → 20
```

⚠ Impure Pipe

Impure pipe: Her change detection cycle'da yeniden çalışır. Çok pahalı.

```
@Pipe({ name: 'filter', standalone: true, pure: false }) // ← false
export class FilterPipe implements PipeTransform {
  transform(items: any[], term: string): any[] {
    return items.filter(i => i.name.includes(term));
  }
}
```

❏ Impure Pipe Tehlikeli

- ✗ Her change detection'da çalışır — saniyede yüzlerce kez
- ✗ Array filter/sort için kullanma — computed kullan
- ✗ HTTP request içeren pipe yazma
- ✗ Date.now() kullanan pipe impure olmalı ama dikkatli

❏ Doğru Alternatif

```
// ❌ Impure pipe
{{ items | filter:term }}

// ✓ Computed signal
@Component({
  template: `
    @for (item of filtered(); track item.id) {
      ...
    }
  `,
})
export class MyComponent {
  items = signal<Item[]>([]);
  term = signal('');

  filtered = computed(() =>
    this.items().filter(i => i.name.includes(this.term()))
  );
  // Otomatik memoize, sadece deps değişince yeniden compute
}
```

Best Practices & Karar Ağacı

Ne zaman pipe, ne zaman computed, ne zaman method?

❑ Karar Ağacı

❑ Hangisini Kullan?

Static dönüşüm (format)? → Pipe
Reactive state derived? → Computed signal
Event response? → Method
Async data? → httpResource / async pipe

❑ Pipe Use Cases

- ✓ Built-in: date, currency, number, async
- ✓ Custom format: truncate, fileSize, timeAgo
- ✓ Sanitization: safeHtml, safeUrl
- ✓ Reusable formatting logic — birden fazla yerde kullanılıyor

❑ Pipe Kullanma

- ✗ Filter/sort: Computed signal kullan
- ✗ HTTP request: resource() kullan
- ✗ Event handler: Method yaz
- ✗ One-time use: Inline expression yaz

⚡ Performance Tips

- ✓ Pipe'lar pure tutulmalı (default)
- ✓ Impure pipe sadece son çare
- ✓ Pipe chain sırası önemli (kısa olan önce)
- ✓ Standalone pipes tercih (tree-shakeable)

❑ Naming Convention

```
// Pipe ismi - camelCase
@Pipe({ name: 'timeAgo' })
@Pipe({ name: 'fileSize' })
@Pipe({ name: 'truncate' })

// Class ismi - PascalCase + Pipe suffix
export class TimeAgoPipe { }
export class FileSizePipe { }
export class TruncatePipe { }
```

Modül 12 Özeti

Pipe'lar template'te veri dönüştürme için güçlü bir araç. Built-in'leri biliyorsun, custom pipe'lar yazabilirsin, pure/impure trade-off'unu anlıyorsun.

Neler Öğrendin?

- ✓ Built-in pipe'lar — date, currency, number, async, slice, json
- ✓ Türkçe locale setup
- ✓ async pipe vs signal — modern yaklaşım
- ✓ Custom pipe yazma — 4 örnek
- ✓ Pure vs Impure pipe — performance impact
- ✓ Pipe vs computed vs method karar verebilmek

Sıradaki Modül

Modül 13 — Directives Derinlemesine. Structural directive'ler, host bindings, directive composition, advanced patterns.